

Plataforma de Gestión de Citas Médicas

(Clínica/Hospital)

Entrega de Avance 2

ASIGNATURA
Programación II

EQUIPO DE TRABAJO
Grupo 7

CATEDRÁTICO
Ing. Daniel Enrique Guevara Gomez

REPOSITORIO
 github.com/NapoSV/medisuite

VIDEO DE SUSTENTACIÓN
 youtu.be/IP1fR6kJY6k

Integrantes del Equipo

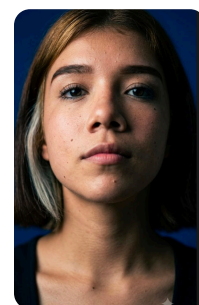
11 MIEMBROS



**MERINO
VENTURA
ALEJANDRO
SEBASTIAN**
CIF: 2026020122
TRABAJÓ



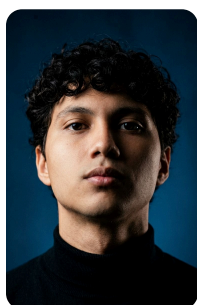
**MELGAR RIVAS
WILLIAM ARIEL**
CIF: 2026011736
TRABAJÓ



**FUENTES ORTIZ
ERIKA ALEXANDRA**
CIF: 2026011709
TRABAJÓ



**ORELLANA ROJAS
BAYRON
ALEXANDER**
CIF: 2026011707
TRABAJÓ



**VENTURA
VELASQUEZ
CARLOS MARIO**
CIF: 2026011585
TRABAJÓ



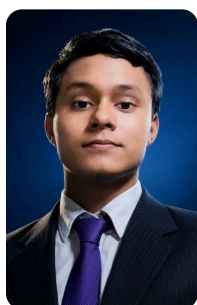
**FLORES
HERNANDEZ
WALTER
ALEJANDRO**
CIF: 2026011012
TRABAJÓ



**SANCHEZ
MENJIVAR NICOLE
NOHEMY**
CIF: 2026010813
TRABAJÓ



**DIAZ SANTOS ZAIR
BENETT**
CIF: 2026010796
TRABAJÓ



**VIGIL RAMIREZ
ALEJANDRO
ANTONIO**
CIF: 2026010204
TRABAJÓ



**LOPEZ RUIZ HECTOR
NAPOLEON**
CIF: 2026010132
TRABAJÓ



**VASQUEZ AMAYA
WALTER AMILCAR**
CIF: 2026010068
TRABAJÓ



CONTENIDO

Índice

01 Introducción

02 Justificación

03 Objetivos

04 Marco Teórico

05 Metodología

06 Historias de Usuario

07 Diseño del Sistema

08 Arquitectura del Sistema

09 Diagrama Entidad-Relación

10 Entradas y Salidas por Historia de Usuario

11 Plan de Pruebas y Resultados

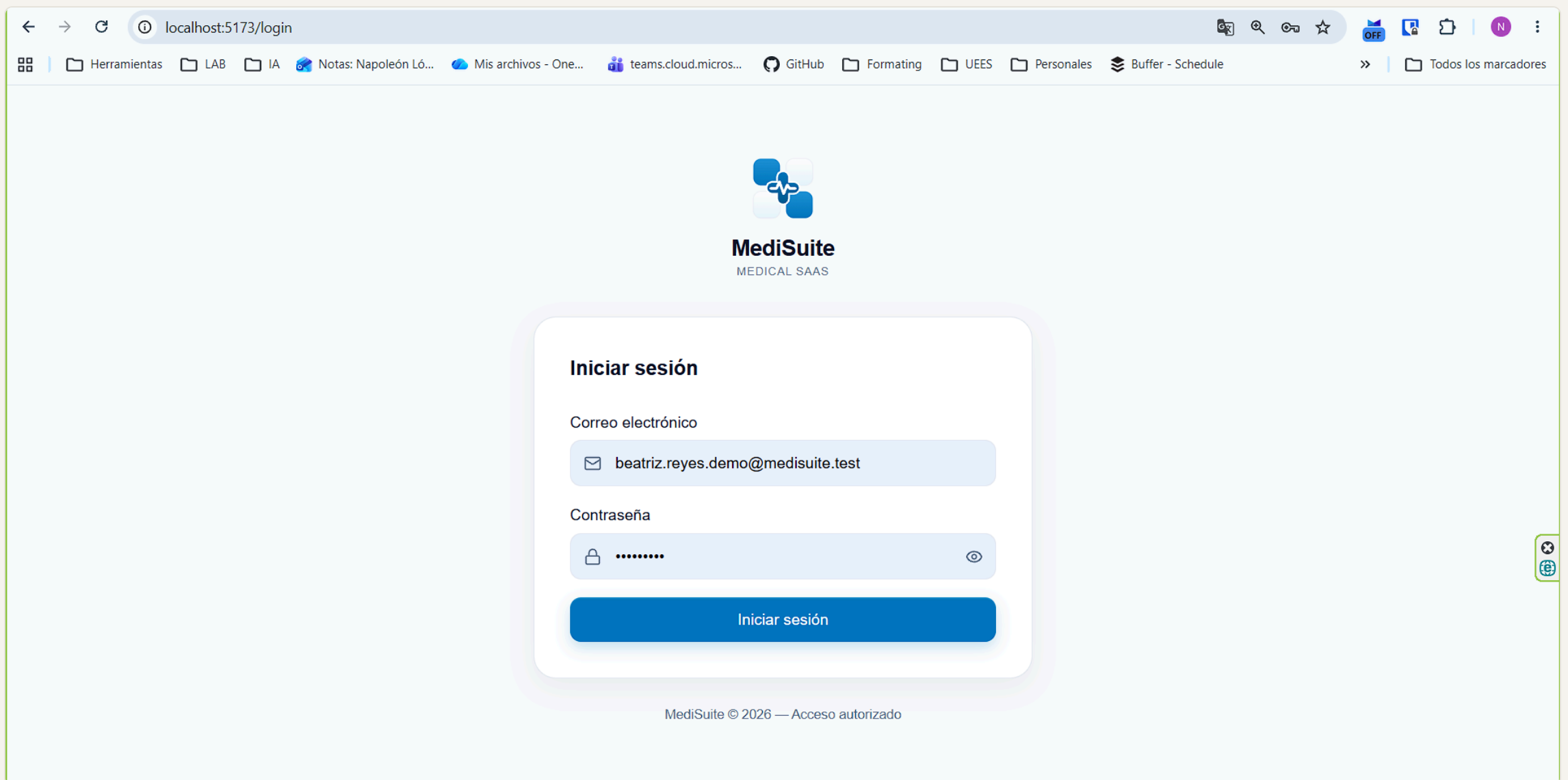
12 El Proyecto - Descripción Técnica

13 Conclusiones

14 Bibliografía

SECCIÓN 01

Introducción



Pantalla de autenticación - selección de clínica, correo y contraseña

MediSuite es un sistema de gestión clínica multi-tenant desarrollado como proyecto integrador de la asignatura Programación II. El sistema permite a múltiples clínicas (tenants) gestionar de forma independiente y segura sus pacientes, médicos, citas, expedientes médicos y recetas, todo sobre una infraestructura compartida con estricto aislamiento de datos por tenant.

El Avance 2 representa la implementación funcional del sistema completo, partiendo de la base arquitectónica establecida en el Avance 1. En esta entrega se integran los módulos de seguridad basada en JWT, gestión de citas con control de concurrencia, expedientes médicos, recetas con impresión A4, un dashboard con métricas en tiempo real y una capa de persistencia híbrida que combina PostgreSQL con archivos binarios `.dat` para respaldo de auditoría.

El desarrollo siguió una metodología ágil Scrum con sprints semanales, integración continua mediante GitHub Actions y revisión de código con Pull Requests obligatorios hacia la rama `develop`.



SECCIÓN 02

Justificación

La digitalización de los procesos clínicos representa una necesidad crítica para las instituciones de salud en El Salvador. Los sistemas actuales en muchas clínicas medianas y pequeñas consisten en registros en papel o hojas de cálculo, lo que genera:

01 Pérdida de historial clínico al extraviar documentos físicos.

02 Imposibilidad de acceso concurrente y remoto al expediente del paciente.

03 Ausencia de trazabilidad en la prescripción de medicamentos.

04 Dificultad para generar reportes y métricas operativas.

MediSuite resuelve estos problemas con una arquitectura moderna, segura y escalable. La decisión de implementar multi-tenancy permite que la misma plataforma sirva a varias clínicas con garantía de aislamiento total de datos, reduciendo el costo de infraestructura respecto a sistemas separados por cliente.

La incorporación de una capa de respaldo en archivos `.dat` (además de PostgreSQL) responde al requisito académico de demostrar persistencia de objetos Java con serialización nativa, integrando los conceptos de concurrencia y persistencia estudiados durante el ciclo.



SECCIÓN 03

Objetivos

3.1 Objetivo General

Desarrollar e integrar los módulos funcionales del sistema MediSuite en su Avance 2, implementando la gestión completa de citas, expedientes médicos, recetas, seguridad JWT multi-tenant y una capa de persistencia híbrida con respaldo en archivos `.dat`.

3.2 Objetivos Específicos

01

Implementar autenticación y autorización basadas en JWT con soporte multi-tenant, incluyendo rate limiting y blacklist de tokens revocados.

02

Desarrollar el módulo de citas con control de concurrencia para evitar conflictos de agenda entre doctores.

03

Crear el expediente médico digital con registro de signos vitales y prescripciones por paciente.

04

Implementar la funcionalidad de recetas médicas con vista de impresión en formato A4.

05

Construir un dashboard con métricas clínicas obtenidas mediante consultas paralelas.

06

Desarrollar una capa de persistencia híbrida que respalda registros de auditoría en archivos `.dat` mediante un scheduler asíncrono.

07

Implementar el frontend en React con todos los módulos operativos y estados de carga consistentes.

08

Garantizar la calidad mediante pruebas unitarias, de integración con Testcontainers y casos manuales.

SECCIÓN 04

Marco Teórico

4.1 Arquitectura Multi-Tenant

El patrón multi-tenant permite que múltiples organizaciones (tenants) compartan la misma instancia de aplicación y base de datos, con aislamiento lógico de datos. MediSuite implementa el enfoque *shared schema* con discriminador `tenant_id` en cada tabla, filtrando automáticamente las consultas mediante el contexto de seguridad del JWT.

4.2 JSON Web Tokens (JWT)

JWT es un estándar abierto (RFC 7519) que define una forma compacta y autocontenida de transmitir información entre partes como un objeto JSON firmado. En MediSuite, el payload del token incluye el identificador del usuario y del tenant, permitiendo resolver ambos contextos sin consultas adicionales a la base de datos en cada request.

4.3 Principios SOLID y Herencia con JPA

La entidad base `BaseEntity` implementa el principio de responsabilidad única al centralizar los campos comunes (`id` , `tenant` , `createdAt` , `updatedAt`) que heredan todas las entidades del dominio. Esto facilita el mantenimiento y garantiza consistencia en el esquema de datos.

4.4 Programación Concurrente en Java

Java proporciona múltiples mecanismos para la programación concurrente. En este proyecto se utilizan dos enfoques complementarios: `ScheduledExecutorService` para la ejecución periódica del respaldo asíncrono a archivos `.dat` , y `CompletableFuture.allOf` para la obtención paralela de métricas del dashboard, reduciendo la latencia de respuesta al ejecutar las consultas de forma simultánea.

4.5 Flyway - Migraciones de Base de Datos

Flyway es una herramienta de control de versiones para esquemas de bases de datos relacionales. Cada cambio en el esquema se codifica como un script SQL versionado (`v1__` , `v2__` , etc.) que Flyway aplica de forma ordenada e idempotente. Esto garantiza que todos los entornos (desarrollo, QA, producción) tengan exactamente el mismo esquema.

SECCIÓN 05

Metodología

El desarrollo del Avance 2 siguió el marco ágil **Scrum** con los siguientes artefactos:

- **Product Backlog:** 88 tareas cargadas en Microsoft Planner, organizadas por módulo y prioridad.
- **Sprints:** 3 sprints semanales (31/08 → 20/09), cada uno con objetivo definido y checkpoint de revisión.
- **Rama de integración:** `develop`. Toda integración requirió Pull Request revisado por el Scrum Master.
- **Integración continua:** GitHub Actions verifica que el build compile correctamente en cada push.
- **Control de calidad:** pruebas unitarias con JUnit 5 + Mockito, integración con Testcontainers y 20 casos manuales documentados.

SPRINT	FECHAS	FOCO PRINCIPAL
S1	31/08 – 06/09	Infraestructura base: BaseEntity, JWT, Flyway, ramas personales
S2	07/09 – 13/09	Módulos core: Citas, Expediente, Dashboard, persistencia <code>.dat</code>
S3	14/09 – 20/09	Integración: Recetas, frontend completo, QA, corrección de bugs

```
backend-1 |      mr1_0.tenant_id,
backend-1 |      mr1_0.updated_at
backend-1 |      from
backend-1 |      medical_records mr1_0
backend-1 | 2026-09-24T04:04:06.775Z  INFO 1 --- [medisuite] [      main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8097 (http
) with context path '/'
backend-1 | 2026-09-24T04:04:06.790Z  INFO 1 --- [medisuite] [      main] c.s.g.medisuite.MediSuiteApplication : Started MediSuiteApplication in 1
1.832 seconds (process running for 12.745)
backend-1 | 2026-09-24T04:04:06.914Z  INFO 1 --- [medisuite] [l-record-backup] c.s.g.m.d.MedicalRecordBackupScheduler : Backup .dat: 3 expedientes respal
dados
backend-1 | 2026-09-24T04:04:09.291Z  INFO 1 --- [medisuite] [nio-8097-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherSer
vlet 'dispatcherServlet'
backend-1 | 2026-09-24T04:04:09.292Z  INFO 1 --- [medisuite] [nio-8097-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherS
ervlet'
backend-1 | 2026-09-24T04:04:09.294Z  INFO 1 --- [medisuite] [nio-8097-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Container medisuite-backend-1 Healthy
frontend-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ 15 not empty, will attempt to perform configuration
```

Docker Compose arrancando los servicios backend y base de datos

SECCIÓN 06

Historias de Usuario

10 historias de usuario implementadas en el Avance 2.

ID	TÍTULO	ROL	PRIORIDAD
HU-001	Autenticación con JWT	Todos los usuarios	ALTA
HU-002	Gestión de pacientes	Recepcionista / Admin	ALTA
HU-003	Gestión de citas	Recepcionista / Doctor	ALTA
HU-004	Expediente médico del paciente	Doctor	ALTA
HU-005	Prescripción de recetas	Doctor	ALTA
HU-006	Dashboard de métricas clínicas	Admin	MEDIA
HU-007	Gestión de doctores	Admin	MEDIA
HU-008	Cambio de contraseña obligatorio en primer login	Todos	MEDIA
HU-009	Respaldo de auditoría en archivo <code>.dat</code>	Sistema	BAJA
HU-010	Vista imprimible de receta médica	Doctor	MEDIA

SECCIÓN 07

Diseño del Sistema

7.1 Diagrama de Casos de Uso

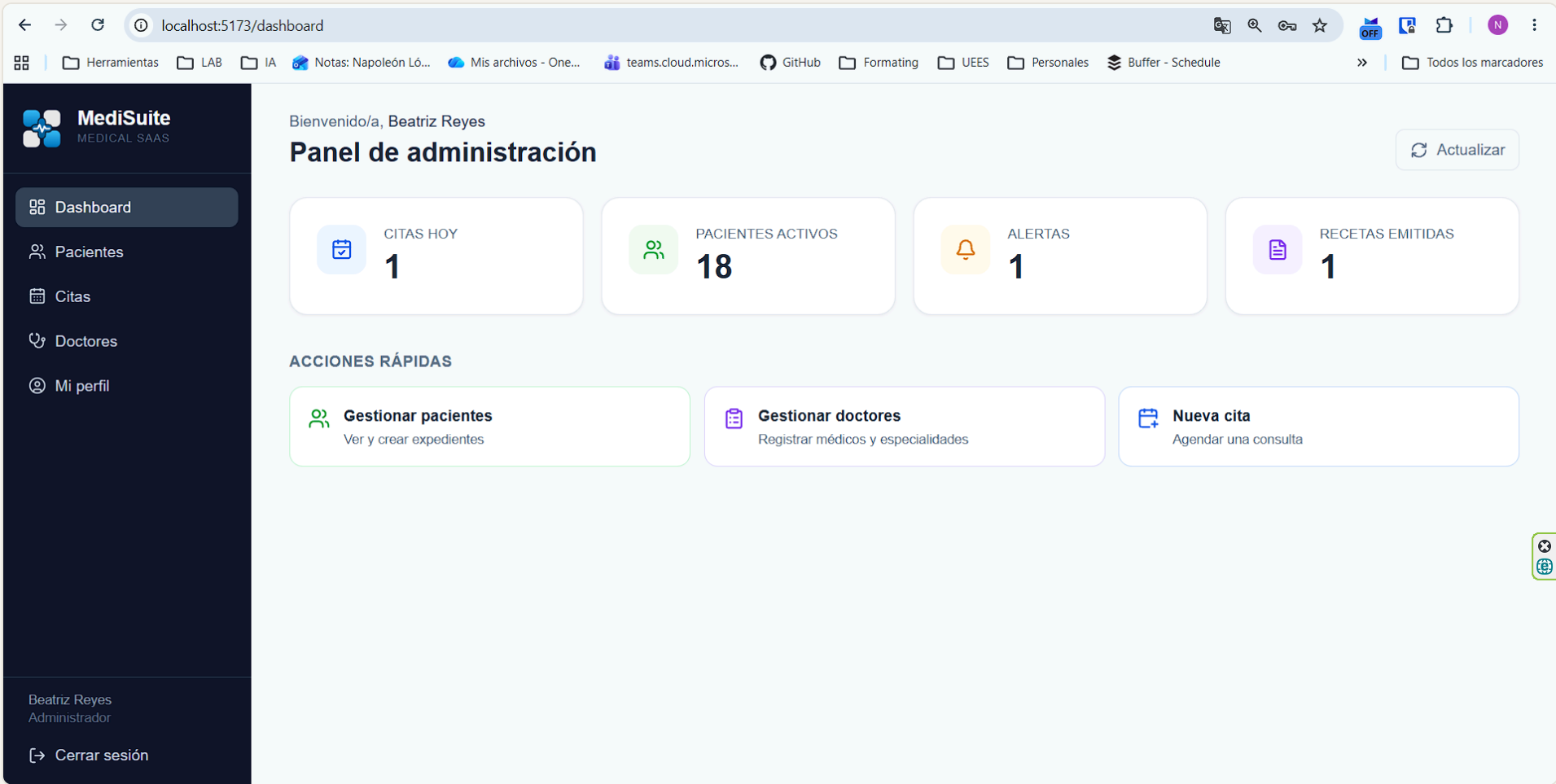
Los actores del sistema son: **Administrador**, **Doctor**, **Recepcionista** y **Sistema** (para procesos automáticos como el respaldo `.dat`).

Administrador
Gestiona usuarios, doctores y tiene acceso a la documentación Swagger.

Doctor
Consulta expedientes, registra signos vitales y emite recetas.

Recepcionista
Registra pacientes, agenda citas y gestiona el estado de las mismas.

Sistema
Ejecuta el scheduler de respaldo cada 60 segundos de forma autónoma.



Dashboard principal con 4 KPIs calculados en paralelo con `CompletableFuture`

7.2 Flujo principal - Ciclo de una cita

Login → Dashboard → Nueva Cita → (seleccionar doctor, fecha, slot)
→ Cita creada (RSV-XXXX) → Paciente llega → Expediente médico
→ Signos vitales → Receta → Impresión A4

SECCIÓN 08

Arquitectura del Sistema

8.1 Visión general

MediSuite sigue una arquitectura en tres capas desacopladas:

CAPA	TECNOLOGÍA	RESPONSABILIDAD
Frontend	React 19 + TypeScript + Tailwind CSS	Interfaz de usuario, consumo REST
Backend	Spring Boot 3.3 + Java 21	Lógica de negocio, seguridad, persistencia
Base de datos	PostgreSQL 16 (Neon serverless)	Almacenamiento relacional multi-tenant

El backend expone una API REST bajo `/api/**`, protegida por un filtro JWT que extrae el `tenant_id` y el `user_id` del token en cada request, publicándolos en el `TenantContext` (ThreadLocal) para su uso transparente en toda la cadena de servicio.

8.2 Estructura de paquetes del backend

```
com.sv.grupo7.medisuite
├─ config/           SecurityConfig, CORS
├─ controller/api/   Controladores REST por módulo
├─ dao/             Repositorios JPA (Spring Data)
├─ dat/             Capa de persistencia .dat (DatFileDao, schedulers)
├─ dto/             Objetos de transferencia
├─ exception/       GlobalExceptionHandler
├─ model/           Entidades JPA (medical/, users/, tenant/, audit/)
├─ security/        JWT, filtros, TenantContext, RateLimitFilter
└─ service/         Lógica de negocio
```

SECCIÓN 09

Diagrama Entidad-Relación

Las entidades principales y sus relaciones son:

```
tenants (1) —< users (N)
tenants (1) —< patients (N)
tenants (1) —< doctors (N)
users (1) — doctors (1) [OneToOne]
doctors (N) >— specialties (1)
patients (1) — medical_records (1)
medical_records (1) —< vital_signs (N)
medical_records (1) —< prescriptions (N)
prescriptions (N) >— doctors (1)
prescriptions (1) —< prescription_items (N)
doctors (1) —< appointments (N)
patients (1) —< appointments (N)
users (1) —< audit_logs (N)
```

—< Uno a muchos (1:N)

>— Muchos a uno (N:1)

— Uno a uno (1:1)

Todas las tablas incluyen los campos heredados de `BaseEntity`: `id` (BIGSERIAL), `tenant_id` (FK a tenants), `created_at` y `updated_at` (TIMESTAMPTZ con zona horaria).

SECCIÓN 10

Entradas y Salidas por Historia de Usuario

HU-001 · HU-002

HU-001

Autenticación con JWT

ENTRADAS

- `tenantSlug` (identificador de la clínica)
- correo electrónico
- contraseña

SALIDAS

- Token JWT firmado con HS256 (payload: `userId` , `tenantId` , `role` , `exp`)
- HTTP 200 con datos del usuario autenticado
- HTTP 401 si las credenciales son incorrectas
- HTTP 429 si se supera el límite de 10 intentos/minuto por IP
- HTTP 423 si la cuenta está bloqueada tras 5 intentos fallidos

HU-002

Gestión de pacientes

MediSuite

Dashboard

Pacientes

Citas

Doctores

Mi perfil

Pacientes

+ Nuevo paciente

Buscar por nombre o DUI (#####-#)

Buscar

Nombre	DUI	Edad	Teléfono	Dirección	
Pedro Hernandez	04141198512345	40 años	7000-3333	Santa Tecla, La Libertad	Editar Ver
Sofía Ramos	04121990001234	36 años	7890-1234	-	Editar Ver
Miguel Solís	04121988005678	38 años	7890-5678	-	Editar Ver
Laura Mendoza	04121995009012	30 años	7890-9012	-	Editar Ver
Carlos Molina	04121992003456	34 años	7891-3456	-	Editar Ver
Ana García	04121987007890	39 años	7891-7890	-	Editar Ver
Roberto Flores	06121998001234	28 años	7892-1234	-	Editar Ver
Carmen Lima	04121996005678	30 años	7892-5678	-	Editar Ver
José Chávez	04121991009012	34 años	7892-9012	-	Editar Ver
Patricia Santos	04121993003456	33 años	7893-3456	-	Editar Ver
Eduardo Torres	04121989007890	37 años	7893-7890	-	Editar Ver
Lucía Martínez	04121985001234	41 años	7894-1234	-	Editar Ver

Lista de pacientes con búsqueda por nombre o DUI

MediSuite

Dashboard

Pacientes

Citas

Doctores

Mi perfil

Expediente: Martha Cáceres

DUI: 01234567-8 Edad: 36 años Tipo de sangre: O+ Alergias: Ninguna conocida

Historial clínico

31/7/2020, 4:56:44 p. m.

Signos vitales

Temp: 36.7°C FC: 72 bpm PA: 120/80 Peso: 68.5 kg Talla: 165 cm

Ninguno

Detalle del paciente - datos clínicos y expediente médico

ENTRADAS

- Nombre, apellido, DUI, fecha de nacimiento, teléfono, dirección, correo (opcional), contacto de emergencia, tipo de sangre, alergias
- Para búsqueda: texto libre (nombre o DUI)

SALIDAS

- Paciente registrado con `id` generado
- Lista paginada de pacientes del tenant activo
- Resultado de búsqueda filtrado
- HTTP 409 si el DUI ya existe en el mismo tenant

SECCIÓN 10

Entradas y Salidas por Historia de Usuario

HU-003

HU-003

Gestión de citas



Dashboard

Pacientes

Citas

Doctores

Mi perfil

Beatriz Reyes

Administrador

Cerrar sesión

Citas

+ Nueva cita

Fecha	Paciente	Doctor	Codigo	Estado	Acciones
24/9/2026, 8:00:00 a. m.	Pedro Hernandez	Ana Martínez	COD-0295	SCHEDULED	Reprogramar Completar Cancelar
1/10/2026, 9:00:00 a. m.	Eduardo Torres	Luis Hernández	RSV-0014	SCHEDULED	Reprogramar Completar Cancelar
2/10/2026, 8:00:00 a. m.	Roberto Flores	José Ramírez	RSV-0027	SCHEDULED	Reprogramar Completar Cancelar
5/10/2026, 8:00:00 a. m.	Pedro Hernandez	María Fuentes	RSV-0021	SCHEDULED	Reprogramar Completar Cancelar
10/10/2026, 9:00:00 a. m.	Patricia Santos	Luis Hernández	RSV-0013	SCHEDULED	Reprogramar Completar Cancelar
15/10/2026, 8:00:00 a. m.	Martha Cáceres	Ana Martínez	COD-3613	SCHEDULED	Reprogramar Completar Cancelar

Módulo de citas - agenda con estado y código de reserva

ENTRADAS

- doctorId , patientId , fecha (YYYY-MM-DD), slot horario (HH:mm), motivo de consulta



Dashboard

Pacientes

Citas

Doctores

Mi perfil

Beatriz Reyes

Administrador

Cerrar sesión

Nueva cita

1. Especialidad

Cardiología

2. Doctor

Luis Hernández (Cardiología)

3. Paciente

Laura Mendoza

Paciente seleccionado: Laura Mendoza - 04121995009012

4. Fecha

30/09/2026

5. Hora disponible

11:00 a. m.

6. Motivo de la consulta

Dolor pectoral izquierdo

Confirmar cita

Modal de nueva cita - selección de doctor, fecha y slot disponible

SALIDAS

- Cita creada con código único (RSV-XXXX), status = PENDING
- Lista de citas del tenant
- Slots disponibles del doctor para una fecha dada
- HTTP 409 si el slot ya está ocupado

SECCIÓN 10

Entradas y Salidas por Historia de Usuario

HU-004

HU-004

Expediente médico del paciente



Dashboard

Mis citas

Pacientes

Triage

Mi perfil

Ana Martínez

Doctor

Cerrar sesión

Expediente: Martha Cáceres

Cerrar triaje

DUI: 01234567-8

Edad: 36 años

Tipo de sangre: O+

Alergias: Ninguna conocida

Registrar signos vitales

Temperatura (°C)

36.8

Frec. cardíaca (lpm)

72

Presión arterial

120/80

Peso (kg)

70.0

Talla (cm)

165

Prioridad

Normal

Síntomas

Describe los síntomas...

Cancelar

Guardar

Historial clínico

31/7/2026, 4:56:44 p. m.

Signos vitales

LOW

Temp: 36.7°C FC: 72 lpm PA: 120/80 Peso: 68.5 kg Talla: 165 cm

"Ninguno"

Expediente médico - historial clínico cronológico de signos vitales y recetas

ENTRADAS

- `patientId` en la URL
- Para triaje: peso (kg), talla (cm), presión arterial, temperatura (°C), frecuencia cardíaca (bpm), síntomas, prioridad (`NORMAL` / `URGENTE` / `EMERGENCIA`)



Dashboard

Mis citas

Pacientes

Triage

Mi perfil

Ana Martínez

Doctor

Cerrar sesión

Triage: Signos vitales

Buscar paciente

Nombre o DUI...

Sofía Ramos

04121990001234 36 años

Temperatura (°C)

36.8

Frec. cardíaca (lpm)

72

Presión arterial

120/80

Peso (kg)

70.0

Talla (cm)

165

Prioridad

Normal

Síntomas / motivo

Describe los síntomas...

Registrar triaje

Últimos registros

Sin registros previos.

Formulario de triaje - registro de signos vitales desde el expediente

SALIDAS

- Expediente completo: datos del paciente, historial cronológico de signos vitales y prescripciones
- Signo vital registrado con `id` generado
- HTTP 404 si el expediente no existe

SECCIÓN 10

Entradas y Salidas por Historia de Usuario

HU-005 · HU-006

HU-005

Prescripción de recetas

MediSuite
MEDICAL SAAS

Dashboard

Mis citas

Pacientes

Triage

Mi perfil

Ana Martínez
Doctor

Cerrar sesión

Nueva receta

Diagnostico

Hipertensión arterial

Medicamentos

+ Agregar medicamento

Medicamento 1

Enalapril 10 mg

1 tableta

Cada 12 horas

Duracion (dias) 30

Indicaciones adicionales

Reducir ingesta de sal. Control en 30 días.

Emitir receta

Formulario de prescripción - ítems dinámicos con medicamento, dosis y frecuencia

ENTRADAS

- `medicalRecordId`, `doctorId` (extraído del JWT)
- Lista de ítems: medicamento, dosis, frecuencia, duración del tratamiento
- El orden de los ítems se preserva mediante `order_idx`

SALIDAS

- Receta creada con `id`, fecha de emisión, lista de ítems ordenada
- HTTP 403 si el usuario no tiene rol de médico

HU-006

Dashboard de métricas clínicas

ENTRADAS

- Token JWT (el `tenantId` se extrae automáticamente del token; no se requieren parámetros adicionales)

SALIDAS

- Objeto JSON con 4 KPIs calculados en paralelo: `appointmentsToday` (citas del día), `activePatients` (pacientes únicos con citas en los últimos 30 días), `pendingAlerts` (citas sin triaje), `prescriptionsIssued` (recetas del día)
- Tiempo de respuesta ≈ latencia de la consulta más lenta

SECCIÓN 10

Entradas y Salidas por Historia de Usuario

HU-007 · HU-008

HU-007

Gestión de doctores

MediSuite
MEDICAL SAAS

Dashboard

Pacientes

Citas

Doctores

Mi perfil

Beatriz Reyes
Administrador

Cerrar sesión

Doctores

+ Nuevo doctor

Nombre	Especialidad	Matricula	Email
Ana Martínez	Medicina General	MED-2020-0456	ana.martinez.demo@medisuite.test
Luis Hernández	Cardiología	MED-2018-0321	luis.hernandez.demo@medisuite.test
María Fuentes	Neurología	MED-2021-0567	maria.fuentes.demo@medisuite.test
José Ramírez	Dermatología	MED-2019-0234	jose.ramirez.demo@medisuite.test
Carlos Doctor	Medicina General	MED-2026-0099	carlos.doctor.test@medisuite.test
Test Doctor	Cardiología	MED-2026-9999	test.doctor2.pw@medisuite.test

Gestión de doctores - lista con especialidad y acciones de administrador

ENTRADAS

- Nombre, apellido, especialidad, número de licencia profesional, horario disponible (JSON: días + bloques horarios)
- Requiere rol ADMIN

SALIDAS

- Doctor creado con `id` y asociado al `User` correspondiente
- Lista de doctores del tenant
- Slots disponibles del doctor para una fecha (`GET /api/appointments/doctors/{id}/slots?date=YYYY-MM-DD`)

HU-008

Cambio de contraseña obligatorio en primer login

MediSuite
MEDICAL SAAS

Dashboard

Pacientes

Citas

Doctores

Mi perfil

Beatriz Reyes
Administrador

Perfil

ADMIN

beatriz.reyes.demo@medisuite.test

Cambiar contraseña

Usa una contraseña segura que no compartas con nadie más.

Contraseña actual

Nueva contraseña

Confirmar nueva contraseña

Cambiar contraseña

Perfil de usuario - cambio de contraseña obligatorio en primer login

ENTRADAS

- Token JWT (con flag `mustChangePassword = true`), contraseña actual, nueva contraseña (mínimo 8 caracteres)

SALIDAS

- Contraseña actualizada con nuevo hash BCrypt
- Campo `must_change_password` puesto en `false`
- HTTP 400 si la contraseña actual no coincide o no cumple longitud mínima

SECCIÓN 10

Entradas y Salidas por Historia de Usuario

HU-009 · HU-010

HU-009 Respaldo de auditoría en archivo .dat

ENTRADAS

- Proceso automático - ninguna entrada del usuario. El `AuditBackupScheduler` se dispara cada 60 segundos mediante `ScheduledExecutorService` configurado con `@PostConstruct`

SALIDAS

- Archivo `data/audit_backup.dat` actualizado con la serialización binaria de todos los `AuditLog` del tenant
- `data/medical_record_backup.dat` con los expedientes médicos
- Log en consola: fecha, hora y cantidad de registros respaldados

HU-010 Vista imprimible de receta médica



Dashboard

Mis citas

Pacientes

Triage

Mi perfil

Ana Martínez

Doctor

Cerrar sesión

Nueva receta

Receta #8 emitida con 1 medicamento(s).

Diagnostico

Medicamentos

+ Agregar medicamento

Medicamento 1

Medicamento (ej. Amoxicilina 500mg)

Dosis (ej. 1 tableta)

Frecuencia (ej. cada 8 horas)

Duracion (dias)

1

Indicaciones adicionales

Emitir receta

ENTRADAS

- `prescriptionId` en la URL
- Acción del usuario: clic en "Imprimir" o `Ctrl+P` desde el navegador

SALIDAS

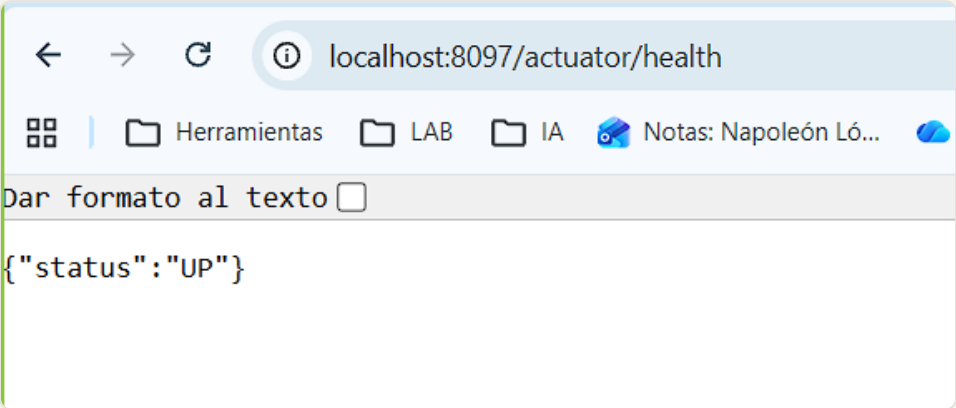
- Página HTML con estilos `@media print` que ocultan navbar y sidebar
- Contenido A4: datos del médico, paciente, fecha de emisión y lista de medicamentos
- `window.print()` abre el diálogo del sistema para imprimir o guardar como PDF

Vista imprimible de receta en formato A4 con estilos `@media print`

Plan de Pruebas y Resultados

```
[INFO] --- surefire:3.2.5:test (default-test) @ medisuite-backend ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sv.grupo7.medisuite.dat.AuditLogDatDaoIntegrationTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.101 s -- in com.sv.grupo7.medisuite.dat.AuditLogDatDaoIntegrationTest
[INFO] Running com.sv.grupo7.medisuite.MediSuiteApplicationTests
WARNING: A Java agent has been loaded dynamically (C:\Users\hlopez\.m2\repository\net\bytebuddy\byte-buddy-agent\1.14.18\byte-buddy-agent-1.14.18.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.292 s -- in com.sv.grupo7.medisuite.MediSuiteApplicationTests
[INFO] Running com.sv.grupo7.medisuite.service.AppointmentServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.276 s -- in com.sv.grupo7.medisuite.service.AppointmentServiceTest
[INFO] Running com.sv.grupo7.medisuite.service.AuthServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.159 s -- in com.sv.grupo7.medisuite.service.AuthServiceTest
[INFO] Running com.sv.grupo7.medisuite.service.MedicalRecordServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.146 s -- in com.sv.grupo7.medisuite.service.MedicalRecordServiceTest
[INFO] Running com.sv.grupo7.medisuite.service.PatientServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.022 s -- in com.sv.grupo7.medisuite.service.PatientServiceTest
[INFO] Results:
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.296 s
[INFO] Finished at: 2026-09-23T22:58:31-06:00
[INFO] -----
(base) PS C:\Users\hlopez\medisuite\backend> |
```

Resultado de mvn test - 19 pruebas ejecutadas, 0 fallos, BUILD SUCCESS



Actuator health endpoint confirmando el estado UP del backend en Docker

11.2 Prueba de integración con Testcontainers

`PatientRepositoryIT` arranca un contenedor real de PostgreSQL 16 Alpine mediante Testcontainers, verificando que los repositorios JPA funcionan con una base de datos real (no mocks). Tres aserciones: contenedor activo, repositorio inyectado, `findAll()` no lanza excepción.

11.3 Test de integración backup .dat

`AuditLogDatDaoIntegrationTest` verifica el ciclo completo de serialización y deserialización del `AuditLogDatDao` : guarda una lista de `AuditLog` en el archivo binario y la recupera, comprobando integridad de los datos.

SECCIÓN 11

Plan de Pruebas y Resultados

11.4 Casos de prueba manuales - Se ejecutaron 20 casos de prueba manuales el 19/09/2026, cubriendo los flujos principales de HU-001 a HU-010. De los 20 casos, 14 pasaron en la primera ejecución; 6 presentaron fallos que fueron documentados y corregidos.

ID	MÓDULO	CASO DE PRUEBA	ESTADO
CP-01	Autenticación	Inicio de sesión correcto	PASS
CP-02	Autenticación	Inicio de sesión sin tenant	PASS
CP-03	Usuarios	Consultar usuario autenticado	PASS
CP-04	Seguridad	Consultar usuario sin autenticación	PASS
CP-05	Pacientes	Listar pacientes	FAIL → CORREGIDO
CP-06	Pacientes	Buscar pacientes	PASS
CP-07	Pacientes	Consultar paciente por ID	FAIL → CORREGIDO
CP-08	Médicos	Listar médicos	FAIL → CORREGIDO
CP-09	Dashboard	Consultar métricas	PASS
CP-10	Citas	Listar citas	PASS
CP-11	Citas	Consultar horarios disponibles	PASS
CP-12	Expediente médico	Consultar expediente de paciente	FAIL → CORREGIDO
CP-13	Recetas	Consultar receta	FAIL → CORREGIDO
CP-14	Autenticación	Cerrar sesión	PASS
CP-15	Seguridad	Acceder después de cerrar sesión	PASS
CP-16	Autenticación	Cambio de contraseña inválida	PASS
CP-17	Autenticación	Login con contraseña incorrecta	FAIL → CORREGIDO
CP-18	Autenticación	Login con correo inválido	PASS
CP-19	Autenticación	Login con contraseña vacía	PASS
CP-20	Seguridad	Dashboard sin autenticación	PASS

SECCIÓN 11

Plan de Pruebas y Resultados

11.1 Pruebas unitarias

Se implementaron pruebas unitarias con JUnit 5 y Mockito para los servicios principales:

CLASE DE PRUEBA	SERVICIO BAJO PRUEBA	CASOS CUBIERTOS
AuthServiceTest	AuthService	Login correcto, contraseña incorrecta, cuenta bloqueada
PatientServiceTest	PatientService	Búsqueda por DUI, paginación, not found
AppointmentServiceTest	AppointmentService	Crear cita, conflicto de slot, cancelar
MedicalRecordServiceTest	MedicalRecordService	Consulta expediente completo

11.5 Resumen de bugs encontrados y corregidos

ID	ENDPOINT	CAUSA RAÍZ	ESTADO
BUG-01	GET /api/patients	LazyInitializationException - falta @Transactional	CORREGIDO
BUG-02	GET /api/patients/{id}	Misma causa raíz	CORREGIDO
BUG-03	GET /api/doctors	LazyInitializationException relaciones lazy	CORREGIDO
BUG-04	GET /api/patients/{id}/medical-record	Serialización de relaciones lazy en Map	CORREGIDO
BUG-05	GET /api/prescriptions/{id}	LazyInitializationException en items	CORREGIDO
BUG-06	POST /api/auth/login (password incorrecta)	Falso positivo de entorno	NO REQUIERE CORRECCIÓN



SECCIÓN 12

El Proyecto - Descripción Técnica

12.1 Infraestructura de Seguridad

La seguridad del sistema se construye en capas, aplicando el principio de defensa en profundidad.

Autenticación con JWT

El endpoint `POST /api/auth/login` valida las credenciales del usuario contra el hash BCrypt almacenado. Al autenticarse correctamente, el servidor genera un token JWT firmado con HS256, cuyo payload incluye el `userId`, el `tenantId` y el rol del usuario. El secreto de firma se valida al arrancar la aplicación para garantizar una longitud mínima de 256 bits (32 bytes UTF-8), cerrando la vulnerabilidad OWASP A02 de secretos débiles.

Filtro JWT (`JwtAuthenticationFilter`)

Intercepta cada request bajo `/api/**`, extrae el token del header `Authorization: Bearer <token>`, lo verifica y publica el `userId` y `tenantId` en el `SecurityContext` y en el `TenantContext` (ThreadLocal). Esto permite que cualquier capa del sistema acceda al tenant actual sin necesidad de pasarlo como parámetro.

Rate Limiting (`RateLimitFilter`)

Implementado con la librería Bucket4j. Limita las solicitudes al endpoint de login a 10 intentos por minuto por IP, devolviendo HTTP 429 al superarse el umbral. Esto previene ataques de fuerza bruta contra la autenticación.

Blacklist de tokens (`JwtBlacklistService`)

Al hacer logout (`POST /api/auth/logout`), el JTI (identificador único) del token se almacena en memoria. El `JwtAuthenticationFilter` consulta la blacklist en cada request, rechazando tokens revocados aunque no hayan expirado aún.

Bloqueo de cuenta

Después de 5 intentos fallidos de login, la cuenta se bloquea por 15 minutos. La operación de incremento es atómica (`UPDATE ... SET sc-camel-failed-login-attempts = failedLoginAttempts + 1`) para prevenir race conditions bajo carga concurrente.

Headers de seguridad OWASP

El `SecurityConfig` configura automáticamente los headers HTTP: `Strict-Transport-Security`, `X-Frame-Options`, `X-Content-Type-Options` y `Referrer-Policy`, protegiendo contra clickjacking, MIME sniffing y downgrade de protocolo.

SECCIÓN 12

El Proyecto - Descripción Técnica

12.2 Multi-Tenancy con Aislamiento por `tenant_id`

El patrón elegido es *shared database, shared schema* con discriminador por columna. Cada tabla del dominio contiene una columna `tenant_id` (FK a `tenants`) que identifica a qué clínica pertenece cada registro.

El `TenantContext` es una clase de utilidad con un `ThreadLocal<Long>` que almacena el `tenant_id` del usuario autenticado durante el ciclo de vida del request. Los servicios obtienen este valor con `TenantContext.currentTenantId()` antes de cualquier operación de escritura.

Las restricciones de unicidad multi-tenant se implementan como índices `UNIQUE (tenant_id, campo)` para garantizar que dos pacientes con el mismo DUI en distintas clínicas sean registros válidos e independientes.

12.3 Capa de Entidades con `BaseEntity`

Todas las entidades del dominio extienden la clase abstracta `BaseEntity`, que centraliza:

- `id`: clave primaria `BIGSERIAL` generada por la base de datos.
- `tenant`: relación `@ManyToOne` lazy hacia `Tenant`, anotada con `@JsonIgnore`.
- `createdAt` / `updatedAt`: marcas de tiempo en `TIMESTAMPZ`, gestionadas con `@PrePersist` y `@PreUpdate`.
- Implementación de `Serializable` con `serialVersionUID`, necesario para la capa `.dat`.

Este diseño aplicó el principio DRY: los campos comunes se escriben una sola vez y se heredan, garantizando consistencia entre las 10+ entidades del sistema.

SECCIÓN 12

El Proyecto - Descripción Técnica

12.4 Control de Versiones de Base de Datos con Flyway

El esquema se gestiona completamente con Flyway. Cada cambio estructural es un script SQL versionado:

VERSIÓN	CONTENIDO
V1	Columna <code>must_change_password</code> en <code>users</code>
V2	Renombrar <code>patients.cif</code> → <code>patients.dui</code>
V3	Restricciones UNIQUE multi-tenant (idempotente con DO-blocks)
V4	Seed de datos demo (5 doctores, 20 pacientes, 30 citas)
V5	Tabla <code>appointments</code> con índice parcial anti-colisión de slots
V6	Tablas <code>medical_records</code> y <code>vital_signs</code>
V7	Refactorización <code>prescriptions</code> y <code>prescription_items</code> con <code>order_idx</code>

La propiedad `spring.jpa.hibernate.ddl-auto=validate` garantiza que Hibernate nunca modifique el esquema automáticamente; solo Flyway tiene autoridad sobre la estructura de la base de datos.

12.5 Módulo de Citas con Control de Concurrencia

Detección de conflictos

El `AppointmentService` verifica antes de crear una cita que el doctor no tenga otra cita en el mismo slot horario con estado distinto a `CANCELLED`. Este chequeo usa una consulta JPQL atómica dentro de una transacción `@Transactional`.

Índice único parcial

La migración V5 define `UNIQUE (doctor_id, scheduled_at) WHERE status <> 'CANCELLED'` como guardia de última línea a nivel de base de datos, impidiendo duplicados incluso bajo carga concurrente.

Endpoint de slots disponibles

GET `/api/appointments/doctors/{doctorId}/slots?date=YYYY-MM-DD`
calcula los slots del `available_schedule` (JSONB) del doctor y descuenta los ya ocupados.

SECCIÓN 12

El Proyecto - Descripción Técnica

12.6 Expediente Médico y Recetas

MedicalRecord

Cada paciente tiene un único expediente (`@OneToOne`). El expediente contiene una colección de signos vitales (`VitalSign`) y prescripciones (`Prescription`),

accesibles mediante

```
GET /api/patients/{id}/medical-record
```

.

Prescriptions

Una receta está asociada a un expediente y un doctor. Contiene una lista ordenada de ítems (

`PrescriptionItem`) usando

```
@OrderColumn(name = "order_idx")
```

para preservar el orden de los medicamentos. Se usa `List` en lugar de `Set` para permitir el mismo medicamento con distintas dosis y garantizar el orden de impresión.

Vista imprimible

La ruta

`/prescriptions/{id}/print` aplica estilos `@media print` que ocultan el navbar y el sidebar, presentando un documento A4 listo para imprimir con `window.print()` .

12.7 Persistencia Híbrida - PostgreSQL + Archivos .dat

POSTGRESQL

PostgreSQL es la fuente de verdad para todos los datos operacionales. Todas las entidades del dominio persisten aquí con integridad referencial y transaccionalidad ACID.

ARCHIVOS .DAT

Archivos .dat son una capa secundaria de respaldo basada en serialización binaria de objetos Java. La clase genérica `DatFileDao<T extends Serializable>` abstrae la lógica usando `ObjectOutputStream` y `ObjectInputStream` . Dos implementaciones concretas:

- `AuditLogDatDao` : respalda los registros de `AuditLog` en `data/audit_backup.dat` .
- `MedicalRecordDatDao` : respalda los expedientes en `data/medical_record_backup.dat` .

SECCIÓN 12

El Proyecto - Descripción Técnica

12.8 Concurrencia - Dos Patrones Implementados

Scheduler asíncrono (AuditBackupScheduler)

Implementado con ScheduledExecutorService mediante @PostConstruct . Al arrancar la aplicación, se inicia un hilo daemon audit-backup que ejecuta el método backup() cada 60 segundos con scheduleAtFixedRate , sin bloquear el hilo principal.

Consultas paralelas con CompletableFuture (DashboardMetricsService)

El endpoint GET /api/dashboard/metrics lanza las cuatro consultas de métricas en paralelo con CompletableFuture.supplyAsync() y espera que todas terminen con CompletableFuture.allOf(...).join() . El tiempo de respuesta total es igual al de la consulta más lenta, no a la suma de todas.

12.9 Frontend - React con TypeScript y Tailwind CSS

La interfaz de usuario fue desarrollada con React 19, TypeScript y Tailwind CSS:

RUTA	COMPONENTE	DESCRIPCIÓN
/login	LoginPage	Formulario de autenticación con manejo de errores
/dashboard	DashboardPage	4 tarjetas KPI con métricas en tiempo real
/patients	Pacientes	Listado con búsqueda por DUI o nombre
/doctors	Doctores	CRUD de doctores (solo ADMIN)
/appointments	Citas	Agenda con tabla y modal de nueva cita
/appointments/new	CitaNueva	Selector de doctor, fecha y slot disponible
/patients/:id/record	Expediente	Timeline del expediente médico del paciente
/prescriptions/new	Recetas	Formulario de receta con ítems dinámicos
/prescriptions/:id/print	RecetaPrint	Vista imprimible A4 con @media print
/profile	Perfil	Datos del usuario con cambio de contraseña

La autenticación usa un AuthContext (useAuth) que persiste el JWT en localStorage y lo adjunta automáticamente a cada request mediante api/client.ts . Las rutas protegidas se implementan con ProtectedRoute , que redirige a /login si no hay token válido.



SECCIÓN 12

El Proyecto - Descripción Técnica

12.10 Pruebas y Aseguramiento de Calidad

Nivel unitario

Pruebas con JUnit 5 y Mockito sobre los servicios de lógica de negocio. Las dependencias externas se simulan con mocks, garantizando que cada prueba evalúe únicamente la lógica del servicio bajo prueba.

Nivel integración

`PatientRepositoryIT` usa Testcontainers para levantar un contenedor real de PostgreSQL 16 durante las pruebas. `AuditLogDatDaoIntegrationTest` verifica el ciclo completo de serialización y deserialización del mecanismo `.dat`.

Nivel sistema (manuales)

20 casos de prueba exploratorios cubriendo los flujos end-to-end de las 10 historias de usuario. Los resultados se documentaron en `docs/fases/casos-de-prueba.md` y los 6 bugs en `docs/fases/reporte-bugs.md`.

La causa raíz común de 5 de los 6 bugs fue la configuración `spring.jpa.open-in-view=false` combinada con métodos de servicio sin `@Transactional` y relaciones JPA lazy. La corrección sistemática consistió en agregar `@Transactional(readOnly = true)` a todos los métodos de lectura y `@JsonIgnore` a las relaciones lazy que Jackson no debía serializar.

12.11 Despliegue con Docker

El sistema se despliega mediante `docker compose` con dos servicios:

backend

Imagen Java 21 (Eclipse Temurin), construida desde el JAR generado por Maven, expuesta en el puerto 8097. Incluye el volumen `backend_data` montado en `/app/data` para persistir los archivos `.dat` entre reinicios del contenedor.

db (opcional en desarrollo)

El ambiente de desarrollo usa la base de datos compartida en Neon (PostgreSQL serverless). En producción local, `docker compose` levanta un contenedor PostgreSQL 16.

Las variables de entorno sensibles (`SPRING_DATASOURCE_URL` , `JWT_SECRET` , `CORS_ALLOWED_ORIGINS`) se configuran en un archivo `.env` que no se versiona. El `docker-compose.yml` referencia estas variables con la sintaxis `${VARIABLE}` , garantizando que ningún secreto quede expuesto en el repositorio.



SECCIÓN 13

Conclusiones

Conclusiones individuales de los integrantes del equipo (1 de 2).

López Ruiz Héctor Napoleón

SCRUM MASTER / ARQUITECTO

El Avance 2 demostró que la arquitectura multi-tenant diseñada en el Avance 1 escala correctamente hacia los módulos más complejos del sistema. La coordinación de 88 tareas entre 11 integrantes mediante GitHub Actions y Pull Requests obligatorios fue clave para mantener la rama `develop` compilable en todo momento. El mayor aprendizaje fue que la seguridad no es un módulo aislado sino un eje transversal: cada decisión técnica (JWT, TenantContext, índices únicos parciales) tiene implicaciones directas en la integridad de los datos.

Vigil Ramírez Alejandro Antonio

BACKEND / SEGURIDAD

La implementación de la capa de seguridad siguiendo los principios OWASP A01–A10 permitió comprender que proteger una API REST requiere decisiones coordinadas en múltiples puntos: el filtro JWT, el rate limiting en el endpoint de login, la blacklist de tokens y los headers HTTP de seguridad. El uso de Bucket4j para rate limiting y la validación del secreto JWT al arrancar la aplicación fueron los mecanismos más importantes para prevenir ataques de fuerza bruta y secretos débiles.

Orellana Rojas Bayron Alexander

BACKEND / MÓDULO CITAS

El módulo de citas fue el más complejo del Avance 2 por los requisitos de concurrencia: dos usuarios simultáneos no pueden reservar el mismo slot. La solución combinó una verificación en el servicio con `@Transactional` y un índice único parcial en PostgreSQL como guardia de última línea. Esta experiencia enseñó que la lógica de negocio en el servicio no es suficiente por sí sola; la base de datos debe ser la fuente de verdad para las restricciones de integridad críticas.

Díaz Santos Zair Benett

FRONTEND

Desarrollar el frontend con React 19, TypeScript y Tailwind CSS en paralelo con el backend demostró la importancia de definir el contrato de la API desde el principio. El manejo consistente de estados de carga y error mediante los componentes `LoadingSpinner` y `ErrorAlert` en todas las páginas fue una decisión que mejoró significativamente la experiencia de usuario. La implementación de rutas protegidas con `ProtectedRoute` y el `AuthContext` como fuente única de verdad para el token JWT simplificaron la lógica de autenticación en el cliente.

Flores Hernández Walter Alejandro

QA / BASE DE DATOS

El trabajo de QA en este avance evidenció que la causa raíz de 5 de los 6 bugs encontrados era la misma: relaciones JPA lazy sin `@Transactional` en los métodos de servicio. Esta consistencia en los fallos permitió aplicar una corrección sistemática y documentar el patrón en el documento de troubleshooting del equipo. Los 20 casos de prueba manuales diseñados cubrieron los flujos end-to-end de las 10 historias de usuario y fueron fundamentales para validar la integración frontend-backend antes del code freeze.

Melgar Rivas William Ariel

FRONTEND

La implementación del diseño responsive con Tailwind CSS demostró que las clases utilitarias permiten adaptar interfaces complejas a diferentes tamaños de pantalla sin escribir media queries personalizadas. El módulo de recetas con ítems dinámicos (agregar y eliminar medicamentos en tiempo real) fue el componente más desafiante del frontend, ya que requirió manejo de estado local cuidadoso para mantener el orden de los ítems sincronizado con el campo `order_idx` del backend.



SECCIÓN 13

Conclusiones

Conclusiones individuales de los integrantes del equipo (2 de 2).

Merino Ventura Alejandro Sebastián

FEATURE CRÍTICO / CONCURRENCIA

La implementación de la capa de persistencia en archivos `.dat` mediante `Dao<T extends Serializable>` integró dos conceptos clave del curso en un solo desarrollo: herencia con clase abstracta genérica y programación concurrente con `ScheduledExecutorService`. El diseño del scheduler asíncrono -que ejecuta el respaldo cada 60 segundos sin bloquear el hilo principal- demostró cómo Java gestiona la concurrencia de forma controlada. El endpoint de dashboard con `CompletableFuture.allOf` redujo el tiempo de respuesta de forma notable al paralelizar las cuatro consultas de métricas.

Fuentes Ortiz Erika Alexandra

QA

La verificación del comportamiento del sistema bajo condiciones de error (credenciales incorrectas, tokens expirados, slots ocupados) fue esencial para garantizar que la aplicación no exponga información sensible en los mensajes de error. Las pruebas de humo en Chrome y Firefox para el flujo de autenticación y la pantalla principal confirmaron que la integración JWT funciona correctamente en el navegador. Este avance consolidó la comprensión de que el QA no comienza al final del sprint sino en paralelo con el desarrollo.

Vásquez Amaya Walter Amílcar

BACKEND / FRONTEND

Trabajar en ambas capas del sistema permitió apreciar el valor del contrato REST como interfaz entre frontend y backend. La implementación de las entradas y salidas por historia de usuario (Sección 10) requirió un análisis detallado de cada endpoint para documentar exactamente qué datos fluyen en cada dirección. Esta tarea evidenció que la documentación técnica no es un complemento del código sino una parte integral del proceso de desarrollo que facilita la integración y el mantenimiento.

Ventura Velásquez Carlos Mario

BACKEND / ENTIDADES

La decisión de usar `List` en lugar de `Set` para los ítems de `Prescription` fue un caso concreto donde la elección del tipo de colección tiene consecuencias directas en el comportamiento del sistema: `Set` no garantiza orden y no permite el mismo medicamento con distintas dosis, mientras que `List` con `@OrderColumn` preserva el orden de ingreso y permite duplicados lógicamente distintos. La implementación de `BaseEntity` como clase abstracta con `Serializable` integró los requisitos académicos de herencia y persistencia de objetos en un solo componente reusable.

Sánchez Menjívar Nicole Nohemy

QA / DOCUMENTACIÓN

La elaboración de la documentación del Avance 2 -que incluye marco teórico, arquitectura, pruebas y conclusiones- permitió comprender que un sistema de software no está completo si no puede ser explicado con claridad. La metodología Scrum con sprints semanales y backlog en Microsoft Planner demostró ser efectiva para un equipo de 11 personas distribuidas, siempre que cada tarea tenga un responsable claro y una fecha de entrega definida. La bibliografía y los estándares de documentación aplicados en este avance serán la base para la entrega final del proyecto.

SECCIÓN 14

Bibliografía

Fuentes consultadas - Formato APA 7ª edición.

1. Docker Inc. (s.f.). *Docker documentation*. Docker. <https://docs.docker.com>
2. Internet Engineering Task Force. (2015). *JSON Web Token (JWT)* (RFC 7519). <https://doi.org/10.17487/RFC7519>
3. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
4. Spring. (s.f.). *Spring Boot reference documentation*. VMware. <https://docs.spring.io/spring-boot/index.html>
5. Tailwind Labs. (s.f.). *Tailwind CSS documentation*. <https://tailwindcss.com/docs>